

1. RAFT Batching transaction model with math example for 5-10 marks

Yes. For **latency per transaction** in a RAFT batching model, the main idea is:

$$T_{\text{transaction}} = T_{\text{batch wait}} + T_{\text{RAFT commit}} \quad T_{\text{transaction}} = T_{\text{batch wait}} + T_{\text{RAFT commit}}$$

Example

Assume:

- Batch size = **5 transactions**
- Transactions arrive every **10 ms**
- RAFT commit time = **30 ms**

Transactions arrive:

$$T_1=0\text{ms}, T_2=10\text{ms}, T_3=20\text{ms}, T_4=30\text{ms}, T_5=40\text{ms} \quad T_1=0\text{ms}, \quad T_2=10\text{ms}, \quad T_3=20\text{ms}, \quad T_4=30\text{ms}, \quad T_5=40\text{ms}$$

The batch becomes full at **40 ms**.

So the waiting time for each transaction is:

Transaction Arrival Batch full Waiting time

T ₁	0 ms	40 ms	40 ms
T ₂	10 ms	40 ms	30 ms
T ₃	20 ms	40 ms	20 ms
T ₄	30 ms	40 ms	10 ms
T ₅	40 ms	40 ms	0 ms

Then the whole batch needs **30 ms** for RAFT replication and majority commit.

Therefore:

$$\begin{aligned} T_1 &= 40 + 30 = 70\text{ms} & T_1 &= 40 + 30 = \boxed{70\text{ms}} \\ T_2 &= 30 + 30 = 60\text{ms} & T_2 &= 30 + 30 = \boxed{60\text{ms}} \\ T_3 &= 20 + 30 = 50\text{ms} & T_3 &= 20 + 30 = \boxed{50\text{ms}} \end{aligned}$$

$$T_4 = 10 + 30 = 40\text{ms} \quad T_4 = 10 + 30 = \boxed{40\text{ms}}$$

$$T_5 = 0 + 30 = 30\text{ms} \quad T_5 = 0 + 30 = \boxed{30\text{ms}}$$

Average latency per transaction

$$T_{\text{avg}} = \frac{70 + 60 + 50 + 40 + 30}{5} = \frac{250}{5} = 50\text{ms} = \boxed{50\text{ms}}$$

★ Important formula for exam

If transactions arrive at a constant interval a , and batch size is B :

$$T_i = (B-i)a + T_{\text{RAFT}} \quad \boxed{T_i = (B-i)a + T_{\text{RAFT}}}$$

and average latency is approximately:

$$T_{\text{avg}} = \frac{(B-1)a^2 + 2aT_{\text{RAFT}} + B^2T_{\text{RAFT}}}{2B} \quad \boxed{T_{\text{avg}} = \frac{(B-1)a^2 + 2aT_{\text{RAFT}}}{2B} + T_{\text{RAFT}}}$$

For $B=5$, $a=10\text{ms}$, $T_{\text{RAFT}}=30\text{ms}$: $B=5$, $a=10\text{ms}$, $T_{\text{RAFT}}=30\text{ms}$:

$$T_{\text{avg}} = \frac{(5-1)(10)^2 + 2(10)(30) + 5^2(30)}{2(5)} = \frac{20 + 60 + 150}{10} = \frac{230}{10} = 23\text{ms} + 30\text{ms} = 53\text{ms} = \boxed{53\text{ms}}$$

In one sentence: Every transaction waits for the batch to fill, then all transactions in that batch experience the RAFT commit time.

2.Steps of Consensus algorithm

Steps of Consensus Algorithm — RAFT

For a **5–10 marks answer**, you can write the steps like this:

1. Leader Election

- All nodes start as **followers**.
- If a follower does not receive a heartbeat, it becomes a **candidate**.
- The candidate requests votes from other nodes.
- If it receives a **majority of votes**, it becomes the **Leader**.

$$\text{Majority} = \left\lfloor \frac{N}{2} \right\rfloor + 1 \quad \text{Majority} = \left\lfloor \frac{5}{2} \right\rfloor + 1 = 2 + 1 = 3$$

For 5 nodes:

$$\text{Majority} = \left\lfloor \frac{5}{2} \right\rfloor + 1 = 2 + 1 = 3$$

2. Client Sends Transaction

The client sends a transaction to the **Leader**.

Example:

$T_1 = \text{Transfer \$100 from A to B}$ $T_1 = \text{Transfer \$100 from A to B}$

The leader receives T_1 .

3. Log Replication

The leader adds the transaction to its log and sends it to the followers.

For example:

$\text{Leader} \rightarrow F_1, F_2, F_3, F_4$ $\text{Leader} \rightarrow F_1, F_2, F_3, F_4$

The followers store the transaction in their logs.

4. Majority Agreement

The leader waits for acknowledgment from a **majority of nodes**.

For 5 nodes:

$\text{Leader} + 2 \text{ Followers} = 3$ $\text{Leader} + 2 \text{ Followers} = 3$

Once 3 nodes have the transaction, consensus is achieved.

5. Commit Transaction

The leader marks the transaction as **committed** and tells followers to commit it.

$T_1: \text{Replicated} \rightarrow \text{Majority} \rightarrow \text{Committed}$ $T_1: \text{Replicated} \rightarrow \text{Majority} \rightarrow \text{Committed}$

6. Apply to State Machine

Finally, every node applies the committed transaction to its state machine/database.

Example:

$\text{Balance}_A = 1000 - 100 = 900$

$\text{Balance}_B = 500 + 100 = 600$

Thus, all healthy nodes eventually reach the **same state**.

Easy Flow to Memorize

Leader Election → Client Request → Log Replication → Majority Agreement → Commit → Apply

$\boxed{\text{Leader Election} \rightarrow \text{Client Request} \rightarrow \text{Log Replication} \rightarrow \text{Majority Agreement} \rightarrow \text{Commit} \rightarrow \text{Apply}}$

Exam conclusion:

A consensus algorithm allows distributed nodes to agree on the same sequence of transactions even when some nodes fail. In RAFT, this is achieved through **leader election, log replication, majority voting, and commitment**.

3. RAFT Failure possibilities

RAFT Failure Possibilities

Failure	What happens?	Can system continue?
Follower fails	Leader continues; remaining followers replicate logs	✓ Yes
Leader fails	Followers detect missing heartbeat and start election	✓ Yes, if majority survives
Multiple followers fail	Remaining nodes continue if majority is available	✓/✗ Depends

Failure	What happens?	Can system continue?
Majority fails	No new transactions can be committed	✗ No
Network partition	The side with majority can continue; minority cannot commit	✓ Majority side
Candidate fails during election	Another election can occur after timeout	✓ Usually
Leader + enough followers fail	No majority remains	✗ No consensus

Example: 5-node RAFT cluster

Nodes:

$\{A, B, C, D, E\} \setminus \{A, B, C, D, E\}$

Majority:

$$\lfloor \frac{5}{2} \rfloor + 1 = 3$$

Case 1 — One follower fails

A(L), B(F), C(F), D(F), E(failed) A(L), B(F), C(F), D(F), E(\text{failed})

There are still **4 active nodes**, so consensus works.

✓ **System continues**

Case 2 — Leader fails

Suppose A is the leader:

A(failed), B, C, D, E A(\text{failed}), B, C, D, E

A new election occurs. Suppose B receives 3 votes:

B → New Leader B \rightarrow \text{New Leader}

✓ **System continues**

Case 3 — Two nodes fail

A(L),B(F),C(F),D(failed),E(failed) A(L),B(F),C(F),D(\text{failed}),E(\text{failed})

Active nodes:

A,B,C=3 A,B,C=3

Since:

$3 \geq \text{Majority}(3)$

✓ **Transactions can still be committed.**

Case 4 — Three nodes fail

A(L),B(F),C(F),D(failed),E(failed) A(L),B(F),C(F),D(\text{failed}),E(\text{failed})

Actually this is only **2 failures**. If **three** fail:

A(L),B(failed),C(failed),D(failed),E(F) A(L),B(\text{failed}),C(\text{failed}),D(\text{failed}),E(F)

Only:

A,E=2 A,E=2

But required:

3

Therefore:

$2 < 3 < 3$

✗ **No new transaction can be committed.**

★ **Key rule to remember**

For **N nodes**:

Maximum tolerable failures = $\left\lfloor \frac{N-1}{2} \right\rfloor$

For 5 nodes:

$$5-1 \div 2 = 2$$

So a **5-node RAFT cluster can tolerate up to 2 node failures** while still making progress.

4. explain PBFT 4 process briefly

PBFT — 4 Processes

PBFT (Practical Byzantine Fault Tolerance) allows distributed nodes to reach agreement even when some nodes may behave maliciously or incorrectly.

It has **4 main phases**:

1. Pre-Prepare

- The **Primary (leader)** receives a client request.
- It assigns a sequence number and broadcasts the proposal to all replicas.

Primary → All Replicas

2. Prepare

- Each replica verifies the proposal.
- If valid, it broadcasts a **Prepare** message to other replicas.

Replica → Other Replicas

A replica needs sufficient matching Prepare messages to move forward.

3. Commit

- Nodes broadcast **Commit** messages after reaching the prepare condition.
- When enough nodes agree, the request is considered committed.

For $3f+1$ nodes, PBFT can tolerate:

f Byzantine failures

Example: For **4 nodes**:

$$3(1)+1=4$$

So 4 nodes can tolerate **1 faulty node**.

4. Reply

- Replicas execute the request.
- They send the result back to the client.
- The client accepts the result after receiving enough matching replies.

★ Easy Flow

Client → Pre-Prepare → Prepare → Commit → Reply

{ Client → Pre-Prepare → Prepare → Commit → Reply }

Remember:

Pre-Prepare = Leader proposes

Prepare = Nodes verify/agrees

Commit = Nodes finalize

Reply = Result sent to client

5. Difference between RAFT and IBFT also RAFT meaning

RAFT Meaning

RAFT stands for **Reliable, Replicated, Redundant, and Fault-Tolerant**.

It is a **consensus algorithm** used in distributed systems to make multiple nodes agree on the same sequence of operations, especially when some nodes may **crash or become unavailable**.

Important: In academic references, Raft is usually described simply as “**Raft**” rather than as a universally standardized acronym. The common paper title is *In Search of an Understandable Consensus Algorithm (Extended Version)*.

RAFT vs IBFT

Feature	RAFT	IBFT
Full form	Raft consensus algorithm	Istanbul Byzantine Fault Tolerance

Feature	RAFT	IBFT
Main purpose	Crash fault tolerance	Byzantine fault tolerance
Fault handled	Nodes crash/stop responding	Nodes can behave maliciously or incorrectly
Leader	Has a leader	Has a proposer/validator
Communication	Relatively simple	More communication between validators
Consensus style	Leader-based log replication	Proposal + voting rounds
Fault tolerance	Handles crash failures	Handles Byzantine failures
Typical use	Private/permissioned distributed systems	Permissioned blockchain
Performance	Generally simpler and faster	More communication overhead
Example	5 nodes can tolerate 2 crashed nodes	$3f+1$ nodes tolerate f Byzantine nodes

Simple Example

Suppose there are **4 nodes**:

A,B,C,D

RAFT

If A is the leader and crashes:

A=failed

The remaining nodes elect a new leader.

RAFT assumes the failed node is generally **not intentionally sending false information**.

IBFT

If A is malicious, it might send different information to different nodes.

IBFT is designed to handle this type of **Byzantine behavior**.

For $f=1$:

$$N=3f+1 \quad N=3f+1 \quad N=3(1)+1=4 \quad N=3(1)+1=\boxed{4}$$

So **4 IBFT validators can tolerate 1 Byzantine faulty validator**.

★ Exam Shortcut

RAFT $\rightarrow$ Crash Fault Tolerance $\boxed{\text{RAFT} \rightarrow \text{Crash Fault Tolerance}}$ IBFT $\rightarrow$ Byzantine Fault Tolerance $\boxed{\text{IBFT} \rightarrow \text{Byzantine Fault Tolerance}}$

RAFT: “A node may fail.”

IBFT: “A node may fail *or behave maliciously*.”

6. Difference between RAFT and PBFT

Difference Between RAFT and PBFT

Feature	RAFT	PBFT
Meaning	Raft consensus algorithm	Practical Byzantine Fault Tolerance
Fault tolerance	Crash faults	Byzantine faults
Malicious nodes	✗ Not designed for malicious behavior	✓ Can tolerate malicious/misbehaving nodes
Leader	One leader is elected	One primary is selected
Consensus method	Leader proposes $\rightarrow$ followers replicate log $\rightarrow$ majority commits	Pre-Prepare $\rightarrow$ Prepare $\rightarrow$ Commit $\rightarrow$ Reply
Fault tolerance	Can tolerate up to $\lfloor (N-1)/2 \rfloor$ crash failures	Requires $N \geq 3f+1$ to tolerate f Byzantine failures
Communication	Relatively low	Higher, because nodes exchange messages with each other

Feature	RAFT	PBFT
Performance	Generally faster and simpler	More computational/communication overhead
Typical use	Distributed databases, replicated services, permissioned systems	Blockchain and systems requiring Byzantine security

Mathematical Example

RAFT — 5 nodes:

$$N=5$$

Maximum crash failures:

$$\lfloor \frac{N-1}{2} \rfloor = \lfloor \frac{5-1}{2} \rfloor = 2$$

So RAFT can continue with **2 failed nodes**.

PBFT — 4 nodes:

$$N=3f+1$$

For $f=1$:

$$N=3(1)+1=4$$

So PBFT can tolerate **1 Byzantine/malicious node**.

★ Easy Exam Difference

RAFT = Crash Fault Tolerance

PBFT = Byzantine Fault Tolerance

In short: RAFT is simpler and faster when nodes are trusted but may crash, while PBFT is designed for environments where some nodes may behave incorrectly or maliciously.

In a PoS blockchain, a block receives attestations from validators controlling 75%
Is the block finalized if deterministic finality requires at least 2/3 of the total stake?
What percentage of stake is required for finalization?
What is meant by deterministic finality and probabilistic finality in PoS?

Answer

Given:

- Validators controlling **75% stake** have attested to the block.
- Deterministic finality requires at least **2/3** of the total stake.

1. What percentage of stake is required for finalization?

$$\frac{2}{3} \times 100 = 66.67\% \quad \frac{2}{3} \times 100 = \boxed{66.67\%}$$

So, **at least 66.67% stake** is required.

Since the block has attestations from **75% stake**:

$$75\% > 66.67\% \quad 75\% > 66.67\%$$

Therefore, **the block is finalized.** ✓

2. Deterministic Finality vs Probabilistic Finality in PoS

Deterministic Finality:

- Once the required validator/stake threshold is reached, the block is **finalized with certainty**.
- It cannot normally be reverted by a valid future chain.
- Example:

$$\text{Required} \geq 66.67\% \quad \text{Required} \geq 66.67\% \quad 75\% \Rightarrow \text{Finalized} \quad 75\% \Rightarrow \boxed{\text{Finalized}}$$

Probabilistic Finality:

- A block becomes **increasingly unlikely to be reversed** as more blocks are added after it.
- It is not immediately guaranteed to be permanent.
- More confirmations → higher confidence in finality.

★ Exam-ready conclusion

In this case, **66.67% (2/3) stake is required for deterministic finality**. Since the block receives attestations from **75% stake**, which is greater than 66.67%, the block achieves deterministic finality. Deterministic finality provides a definite guarantee, whereas probabilistic finality provides increasing confidence as more confirmations are added.

7. Consider a private blockchain network running the Raft consensus protocol with 5 nodes (N1, N2, N3, N4, N5). [4] Assume, Node N3 is currently the elected Leader. Suddenly, N3 crashes. Explain the step-by-step transition process that Node N4 goes through to become the new leader, including the condition that triggers this change

Raft Leader Election — Step-by-Step

Given:

- **5 nodes:** N1,N2,N3,N4,N5
- **N3 is the current Leader**
- N3 suddenly **crashes**

Step 1: Leader Failure

Normally, the leader periodically sends **heartbeat messages** to followers.

N3 → N1, N2, N4, N5

After N3 crashes, these heartbeats stop.

Step 2: Election Timeout

N4 waits for a predefined **election timeout**.

If N4 does not receive a heartbeat from N3 before the timeout expires:

Election Timeout ⇒ Start Election

This is the **condition that triggers the change**.

Step 3: N4 Becomes Candidate

N4 changes its state:

Follower $\rightarrow$ Candidate $\boxed{\text{Follower} \rightarrow \text{Candidate}}$

It then:

- Increments its **term number**.
- Votes for itself.
- Sends a **RequestVote** message to the other nodes.

N4 $\rightarrow$ N1, N2, N5: RequestVoteN4 $\rightarrow$ N1, N2, N5: \ RequestVote

Step 4: Other Nodes Vote

If N1, N2, and N5 consider N4's log sufficiently up-to-date and have not already voted in that term, they can vote for N4.

For 5 nodes, majority is:

$$\left\lfloor \frac{5}{2} \right\rfloor + 1 = 3$$

N4 already has its own vote:

$$N4 + N1 + N2 = 3$$

Therefore, N4 has a **majority**.

Step 5: N4 Becomes Leader

After receiving votes from a majority:

N4: Candidate $\rightarrow$ Leader $\boxed{N4: \text{Candidate} \rightarrow \text{Leader}}$

N4 starts sending **heartbeat messages** to N1, N2, and N5.

N4 $\rightarrow$ N1, N2, N5: HeartbeatN4 $\rightarrow$ N1, N2, N5: \ Heartbeat

This prevents another election from starting.

★ Exam Flow

N3 Crash→No Heartbeat→N4 Election Timeout→N4 Candidate→RequestVote→Majority Votes→N4 Leader
 $\boxed{\text{N3 \ Crash \rightarrow No \ Heartbeat \rightarrow N4 \ Election \ Timeout \rightarrow N4 \ Candidate \rightarrow RequestVote \rightarrow Majority \ Votes \rightarrow N4 \ Leader}}$

Key condition:

N4 becomes a candidate when it does not receive a heartbeat from the current leader N3 within the election timeout. It becomes the new leader after receiving votes from a **majority (at least 3 out of 5 nodes)**.

8.. What happens when the longest version of a blockchain starts spreading across the network? What specific name is given to a block that gets discarded when the network adopts a longer chain? [3]

9. What two main sections make up each block? List the specific items contained inside the Block Header.

8.Longest Chain and Discarded Block [3]

When the **longest/valid chain** starts spreading through the network, nodes eventually adopt it as the main blockchain because it has more accumulated work/weight according to the consensus rules.

A block that was valid but is no longer part of the adopted main chain is called an:

Orphan Block
 $\boxed{\text{Orphan Block}}$

It can occur when two miners/validators produce competing blocks at nearly the same time and the network later chooses one chain.

Exam answer:

When the longest valid blockchain spreads, nodes switch to it and discard the competing shorter branch. The discarded block is commonly called an **orphan block** (more precisely, a stale block in Bitcoin terminology).

9. Two Main Sections of a Block [3]

Each blockchain block generally consists of **two main sections**:

1. **Block Header**
2. **Block Body / Block Data**

Block Header contains:

For a typical Bitcoin-style block:

- **Version** — specifies the block/version rules.
- **Previous Block Hash** — hash of the previous block.
- **Merkle Root** — summarizes all transactions in the block.
- **Timestamp** — time associated with block creation.
- **Difficulty Target (nBits)** — target used for Proof-of-Work.
- **Nonce** — value miners change to find a valid block hash.

Easy to memorize:

Header=Version+Previous Hash+Merkle Root+Timestamp+Difficulty+Nonce
boxed{ Header = Version + Previous\ Hash + Merkle\ Root + Timestamp +
Difficulty + Nonce }

Block Body mainly contains the **transactions/data** included in that block.

10. A software development company contracts a freelance developer to build a mobile application. A smart contract holds a total payment of 100 Tokens, divided into two equal milestone payouts of 50 Tokens each. [4] Milestone 1: The developer submits the design prototype; once approved by the company, 50 Tokens are released automatically. Milestone 2: The developer submits the final application code; once approved, the remaining 50 Tokens are released. a) Describe the step-by-step state transitions of the smart contract from order creation to final payout. What conditions must be satisfied before Milestone 1 funds can move from the contract's balance to the developer's address? b) The company approves Milestone 1, and the smart contract automatically transfers the 50 Tokens to the developer. Which two smart contract features are demonstrated by this process? Explain briefly.

Smart Contract Milestone Payment [4 Marks]

(a) State transitions and conditions

The smart contract contains **100 Tokens**, divided into two milestones:

$$100 = 50 + 50$$

Step-by-step state transition:

1. Order Creation

The company and developer agree on the project terms.

ContractCreated\boxed{ContractCreated}

The company deposits **100 Tokens** into the smart contract.

2. Milestone 1 Submitted

Developer submits the **design prototype**.

M1_Submitted\boxed{M1_Submitted}

The contract waits for approval from the company.

3. Milestone 1 Approved

The company verifies and approves the prototype.

M1_Approved\boxed{M1_Approved}

The smart contract condition is now satisfied, so it automatically releases:

50 Tokens $\rightarrow$ Developer 50 Tokens $\rightarrow$ Developer

4. Milestone 2 Submitted

Developer submits the **final application code**.

M2_Submitted\boxed{M2_Submitted}

5. Milestone 2 Approved and Final Payout

Company approves the final code.

M2_Approved\boxed{M2_Approved}

The remaining:

50 Tokens $\rightarrow$ Developer 50\ Tokens \rightarrow Developer

Contract reaches:

Completed\boxed{Completed}

Conditions for Milestone 1 payment

The 50 Tokens can move from the contract to the developer only when:

1. **The contract has sufficient balance (100 Tokens deposited).**
2. **Developer has submitted the Milestone 1 prototype.**
3. **The company has approved Milestone 1.**
4. The smart contract's predefined payment conditions are satisfied.

Therefore:

M1 Submitted + Company Approval + Sufficient Balance $\Rightarrow$ 50 Tokens Released\boxed{M1_Submitted + Company_Approval + Sufficient_Balance \rightarrow 50\ Tokens_Released}

(b) Two smart contract features demonstrated

1. Automation / Self-Execution

Once the company approves Milestone 1, the contract **automatically executes the payment** without requiring another manual transaction.

Approval $\Rightarrow$ Automatic Payment Approval \rightarrow Automatic_Payment

2. Conditional Execution

The payment occurs only when the predefined condition is satisfied:

Company Approval=True\boxed{Company\ Approval = True}

If the company does not approve the milestone, the 50 Tokens remain locked in the contract.

★ Short exam answer

The process demonstrates **automation (self-execution)** and **conditional execution**. The smart contract automatically transfers 50 Tokens when the predefined condition—company approval of Milestone 1—is satisfied, without requiring a third-party intermediary.

11. In a dApp marketplace, a user clicks "Buy". The dApp needs to call a smart contract function. The user must approve/sign the transaction using their wallet. The transaction is then sent to the blockchain through an RPC provider. [2] a) Write the flow of components involved in communication from user to the blockchain networks. b) Identify one component that handles user interaction and one component that contains the business rules. Why does the user need to sign the transaction? 3. A university deploys a smart contract to manage student certificates. After deployment, the university discovers that there is a mistake in the certificate validation logic. The administrator tries to directly edit the deployed smart contract code. [2] a) Can the deployed smart contract normally be directly modified? Which feature of smart contracts explains your answer? b) Suggest one approach that could be used to introduce the corrected logic. 4. A manufacturing company wants to maintain a blockchain for its internal supply-chain records. Only employees and authorized departments of the company should participate. The company requires fast transaction processing and does not want outsiders to access the network. [2] a. Which blockchain should the company select? b. Name one suitable consensus mechanism that fit for selected blockchain.

dApp Marketplace [2 Marks]

(a) Communication flow

User→dApp/Frontend→Wallet→RPC Provider→Blockchain Network→Smart Contract\boxed{ User \rightarrow dApp/Frontend \rightarrow Wallet \rightarrow RPC\ Provider \rightarrow Blockchain\ Network \rightarrow Smart\ Contract }

Example: User clicks **Buy** → dApp calls contract → wallet asks user to sign → signed transaction goes through RPC → blockchain executes the smart contract function.

(b) Components and signing

- **User interaction: dApp / Frontend** — provides the Buy button and interacts with the user.
- **Business rules: Smart Contract** — contains the buying/payment logic.

Why sign?

The user signs to **authenticate and authorize the transaction using their private key**. It proves that the transaction was approved by the wallet owner and prevents someone else from submitting transactions on their behalf.

3. Smart Contract Modification [2 Marks]

(a) Can it normally be directly modified?

No. A deployed smart contract is normally **immutable**, meaning its deployed code cannot be directly edited or changed.

Smart Contract → Immutable after deployment

This immutability provides trust and ensures that the contract's rules cannot be secretly changed.

(b) How can corrected logic be introduced?

One approach is to:

Deploy a new corrected contract

Then migrate/redirect the application and relevant data/assets to the new contract.

Alternative: If the original contract was specifically designed with an **upgradeable proxy pattern**, its implementation can be upgraded.

4. Manufacturing Company's Blockchain [2 Marks]

(a) Which blockchain?

Private (Permissioned) Blockchain\boxed{\text{Private (Permissioned) Blockchain}}

Because only **employees and authorized departments** should participate, and outsiders should not have access.

(b) Suitable consensus mechanism

RAFT\boxed{\text{RAFT}}

RAFT is suitable for a trusted/permissioned internal network because it provides **fast consensus with relatively low communication overhead**.

Final answer:

Private/Permissioned Blockchain + RAFT consensus is suitable for the company's internal supply-chain system because participants are known and authorized, while fast transaction processing is required.

CT SOLVE

1. Electricity Cost of Re-mining [3]

There are **80 blocks**, and the attacker modifies **Block 25**.

The attacker must re-mine the modified block and **all blocks after it**:

25, 26, ..., 80

Number of affected blocks:

$$80 - 25 + 1 = 56 \quad \boxed{56}$$

Energy required per block:

$$0.5 \text{ kWh} \quad \boxed{0.5 \text{ kWh}}$$

Total energy:

$$56 \times 0.5 = 28 \text{ kWh} \quad \boxed{28 \text{ kWh}}$$

Electricity cost:

$$28 \times \text{TK } 10 = \text{TK } 280 \quad 28 \times \text{TK } 10 = \boxed{\text{TK } 280}$$

✓ **Answer: TK 280**

Genesis

Initial record

Previous hash 000000

Block hash 7e0021

Block 1

Alice pays Bob 4 coins

Previous hash 7e0021

Block hash 6b70c5

Block 2

Bob pays Chen 9 coins

Previous hash 6b70c5

Block hash 9ec243

Block 3

Chen pays Dana 1 coins

Previous hash 4cfc45

Block hash 8e317e

Block 2 changed: its hash and every later link are invalid.

Tampered block

—123

—123

Give feedback

2. Probability of 35% Pool Mining at Least One of Next Two Blocks

Hash power of the pool:

$$p=35\%=0.35$$

Probability it **does not** mine a block:

$$1-p=1-0.35=0.65$$

Probability it does **not mine either** of the next two blocks:

$$0.65 \times 0.65 = 0.4225$$

Therefore, probability of mining **at least one**:

$$P(\geq 1) = 1 - P(0) = 1 - 0.4225 = 0.5775 = \boxed{0.5775}$$

As a percentage:

$$57.75\% = \boxed{57.75\%}$$

✓ **Answer: 57.75%**

3. A user initiates a transaction in a blockchain network. Trace its complete lifecycle from transaction creation to final confirmation, mentioning the role of nodes, mempool, miners, block validation, and consensus. A blockchain has 120 blocks. If Block 30 is modified and re-mining each affected block consumes 0.25 kWh at TK 8/kWh, calculate the total electricity cost of rebuilding the affected portion of the blockchain. A blockchain has mining pools with hash powers of 45%, 35%, and 20%. What is the probability that the 20% pool mines at least one of the next two blocks

1. Transaction Lifecycle in a Blockchain

The complete lifecycle can be explained in these steps:

Step 1: Transaction Creation

The user creates a transaction, for example:

$$A \rightarrow B: 10 \text{ Coins}$$

The transaction is digitally **signed using the user's private key**.

Step 2: Transaction Broadcast

The signed transaction is sent to a nearby **blockchain node**.

User → Node

The node checks basic validity, such as the signature, balance, and transaction format.

Step 3: Mempool

If valid, the transaction is placed in the node's **mempool**, which is a temporary waiting area for unconfirmed transactions.

Transaction → Mempool

Nodes also propagate the transaction to other nodes.

Step 4: Miner Selects Transaction

The **miner** selects transactions from its mempool and creates a candidate block.

Mempool → Miner → Candidate Block

Step 5: Consensus

Depending on the blockchain, the consensus mechanism determines which block is accepted.

For **Proof of Work (PoW)**, the miner solves a computational puzzle.

For **Proof of Stake (PoS)**, validators are selected to propose/attest to blocks.

Step 6: Block Validation

Other nodes receive the proposed block and verify:

- Block structure
- Transactions
- Digital signatures
- Previous block hash
- Consensus requirements

If valid, nodes accept and propagate the block.

Step 7: Confirmation

The block is added to the blockchain.

Transaction → Mempool → Mining/Validation → Consensus → Block → Confirmation
 $\boxed{\text{Transaction} \rightarrow \text{Mempool} \rightarrow \text{Mining/Validation} \rightarrow \text{Consensus} \rightarrow \text{Block} \rightarrow \text{Confirmation}}$

As additional blocks are added, the transaction generally gains more confirmations/finality depending on the blockchain's consensus model.

2. Electricity Cost of Rebuilding Blockchain

Given:

- Total blocks = **120**
- Modified block = **Block 30**
- Energy per affected block = **0.25 kWh**
- Electricity cost = **TK 8/kWh**

Affected blocks are:

30, 31, ..., 120

Number of affected blocks:

$$120 - 30 + 1 = 91 \quad \boxed{91}$$

Total electricity:

$$91 \times 0.25 = 22.75 \text{ kWh}$$

Total cost:

$$22.75 \times 8 = \text{TK } 182 \quad \boxed{\text{TK } 182}$$

✓ **Answer: TK 182**

3. Probability of 20% Pool Mining at Least One of Next Two Blocks

Hash power:

$$p=20\%=0.20$$

Probability of **not** mining one block:

$$1-p=0.80$$

Probability of not mining **either** of the next two blocks:

$$0.80^2=0.64$$

Therefore, probability of mining **at least one**:

$$P(\geq 1) = 1 - P(0) = 1 - 0.64 = 0.36 = \boxed{0.36}$$

As a percentage:

$$36\%$$

4. A user initiates a transaction in a blockchain network. Trace its complete lifecycle from transaction creation to final confirmation, mentioning the role of nodes, mempool, miners, block validation, and consensus. Four mining pools have hash powers of 40%, 30%, 20%, and 10%. What is the probability that the 20% pool mines the next three blocks consecutively? What does this indicate about the role of hash power in PoW consensus? A Proof-of-Work blockchain has a difficulty target of 2^{220} . A miner generates hashes uniformly over a 256-bit hash space using the SHA-256 hash function. What is the approximate probability that a single hash attempt satisfies the difficulty target? Why can pointers in a traditional linked list not directly connect blocks stored on different machines in a distributed network? How do cryptographic hashes solve this problem?

1. Complete Lifecycle of a Blockchain Transaction

Step 1: Transaction Creation

The user creates a transaction and signs it using their **private key**.

User $\rightarrow$ Transaction

Example:

A→B:10 BTCA \rightarrow B: 10\ BTC

Step 2: Broadcast to Nodes

The signed transaction is sent to a nearby **node**. The node verifies basic things such as the digital signature, balance, and transaction format. If valid, it broadcasts the transaction to other nodes.

Step 3: Mempool

Valid but unconfirmed transactions are stored in the node's **mempool**.

Transaction→MempoolTransaction \rightarrow Mempool

The mempool acts as a waiting area before transactions are included in a block.

Step 4: Miner Selects Transactions

A **miner** selects transactions from its mempool and creates a candidate block.

Mempool→Miner→Candidate BlockMempool \rightarrow Miner \rightarrow Candidate\ Block

Step 5: Consensus

In a PoW blockchain, miners compete to solve the Proof-of-Work puzzle. The miner that successfully finds a valid block broadcasts it to the network.

Step 6: Block Validation

Other nodes verify the block, including:

- Transactions and signatures
- Previous block hash
- Block structure
- Proof-of-Work

If valid, nodes accept and propagate the block.

Step 7: Confirmation

The accepted block is added to the blockchain, making the transaction **confirmed**. Additional blocks provide further confirmations.

Creation→Broadcast→Mempool→Mining→Consensus→Validation→Confirmation
 $\boxed{\text{Creation} \rightarrow \text{Broadcast} \rightarrow \text{Mempool} \rightarrow \text{Mining} \rightarrow \text{Consensus} \rightarrow \text{Validation} \rightarrow \text{Confirmation}}$

2. Probability of 20% Pool Mining 3 Consecutive Blocks

Hash power of the pool:

$$p=20\%=0.20$$

For one block:

$$P=0.20$$

For **three consecutive blocks**, assuming each block's outcome is independent:

$$P=0.20 \times 0.20 \times 0.20 \quad P=0.20^3 \quad P=0.008$$

As a percentage:

$$0.8\%$$

What does this indicate?

In PoW, a mining pool's probability of finding a block is approximately proportional to its **hash power**.

So:

Higher hash power $\Rightarrow$ Higher probability of mining blocks
 $\boxed{\text{Higher hash power} \rightarrow \text{Higher probability of mining blocks}}$

However, even a 20% pool can occasionally mine several blocks consecutively; the probability here is **0.8%**.

3. Probability of a Hash Satisfying the Difficulty Target

Given:

- SHA-256 output = **256 bits**

- Total possible hash values:

$$2^{256}$$

- Difficulty target:

$$2^{220}$$

A hash is successful when:

$$\text{Hash} \leq \text{Target}$$

Therefore, approximate probability for one hash attempt is:

$$P = \frac{2^{220}}{2^{256}} = 2^{220-256} = 2^{-36}$$

Numerically:

$$2^{-36} \approx 1.455 \times 10^{-11}$$

Therefore:

$$P \approx 0.0000000001455$$

or approximately:

$$1.455 \times 10^{-9}\%$$

Interpretation

A single hash has an extremely small probability of satisfying the target, so miners must perform a very large number of hash attempts.

4. Why Traditional Linked-List Pointers Cannot Connect Distributed Blocks

In a traditional linked list, a node contains a **memory address/pointer** to the next node:

Block A → Memory Address of Block B
 $\text{Block_A} \rightarrow \text{Memory Address of Block_B}$

This works when both blocks are stored within the same machine/address space.

But in a distributed blockchain, Block A and Block B may be stored on **different computers**.

The memory address of Block B on Machine 2 is not directly meaningful to Machine 1.

Therefore, a traditional pointer cannot reliably link blocks across different machines.

How Cryptographic Hashes Solve This

Instead of storing the physical memory address of the previous block, a blockchain stores the **cryptographic hash of the previous block**.

Block₂ → Hash(Block₁) Block₂ → Hash(Block₁)

For example:

Block₁ → Hash H₁ Block₁ → { Hash } H₁

Then:

Block₂ = [Data, H₁] Block₂ = [Data, H₁]

Every node can independently calculate:

Hash(Block₁) Hash(Block₁)

and compare it with the hash stored in Block 2.

Therefore, the blocks can be logically connected **without needing a physical memory pointer**.

★ Exam Summary

Traditional pointers depend on machine-specific memory addresses and therefore cannot directly reference blocks stored on different machines. Cryptographic hashes provide a machine-independent link: each block stores the hash of the previous block. This creates a tamper-evident **hash chain** across the distributed network.

in proof of stake what is Dpos and Npos describe briefly

DPoS and NPoS in Proof of Stake

1. DPoS — Delegated Proof of Stake

In **DPoS**, token holders **vote for a small group of delegates/block producers** who validate transactions and create blocks on behalf of the network.

Process:

Token Holders → Vote → Delegates → Create Blocks
 $\text{Token Holders} \rightarrow \text{Vote} \rightarrow \text{Delegates} \rightarrow \text{Create Blocks}$

Example:

If 100 people hold tokens, they may vote for **21 delegates**. Those 21 delegates take turns producing blocks.

Advantages: Fast and scalable, but it is more centralized because only a limited number of delegates produce blocks.

2. NPoS — Nominated Proof of Stake

In **NPoS**, token holders can **nominate validators** by backing them with their stake. The protocol selects a set of validators based on the nominations and stake.

Process:

Nominators → Nominate/Back Validators → Selected Validators → Validate Blocks
 $\text{Nominators} \rightarrow \text{Nominate/Back Validators} \rightarrow \text{Selected Validators} \rightarrow \text{Validate Blocks}$

The nominators typically **do not vote for a fixed small group in the same way as DPoS**; they support validators, and the protocol determines the active validator set.

★ Main Difference

DPoS	NPoS
Users vote for delegates	Users nominate/back validators

DPoS

Small elected group produces blocks

More directly election-based

Example: EOS

NPoS

Protocol selects an active validator set

More stake/nomination-based

Example: Polkadot

Easy memory:

DPoS=Delegate\boxed{DPoS = Delegate} NPoS=Nominate\boxed{NPoS =
Nominate}